Advanced Algorithms
CSC902

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

Table of Contents

1. Overview of the course
2. Recap: Algorithms
3. Algorithm Design Paradigms: Divide and Conquer
   - Lower bounds on Comparison-based sorting
   - Finding i^{th} smallest number
   - D & C: Long Integer Multiplications
4. References

Overview of the course

CSC902: Advanced Algorithms

- Revisit Algorithm design paradigms-divide and conquer, greedy and dynamic programming
- Flow Network and Matching
- String Matching
- Computational geometry
- Advanced Data structures: Splay tree, Tries, Suffix tree
- Randomized data structure: Skip List
- Randomized Algorithms: randomized Quicksort, Select, Closest pair
- NP-Completeness: Proving NP-Completeness
- Approximation Algorithms: Vertex cover, TSP, Bin packing,k-center approximation

CSCP901: Advanced Algorithm Laboratory: Implementations of algorithms covered in class

Table of Contents

1. Overview of the course
2. Recap: Algorithms
3. Algorithm Design Paradigms: Divide and Conquer
   - Lower bounds on Comparison-based sorting
   - Finding i^{th} smallest number
   - D & C: Long Integer Multiplications
4. References

Algorithm

Any well-defined computational procedure that takes some values as **input** and produces some values as **output**. Any algorithm satusfies the following:

- Inputs: It should have zero or more inputs
- Outputs: It should produce atleast one output
- Finiteness: Each step must terminate in finite time
- Definiteness: Each instruction/step must be unambiguous and precise
- Effectiveness: Each step must be sufficiently basic enough that can indeed be computed exactly in finite amount of time.

Issues in Algorithm Design

- **Correctness:** Need to prove the corretness of algorithm , typically using an *invariant*, that is, a logical assertion about the state of the data at a given point in the algorithm.
- **Efficiency:** Time and space complexity (a function of input size n) of an algorithm need to be obtained in order to justify the efficiency of that algorithm
  - **Worst case complexity**: Among all inputs of the same size, what is the maximum running time?
  - **Average case complexity**:Among all inputs of the same size, what is the average running time? computed assuming some probability distribution that describes the likelihood that a particular input will arise.
- **Presenting an Algorithm:** Give the Pseudocode or steps, prove correctness and compute time and space complexity.

Asymptotic Notations (1)

Asymptotic Analysis

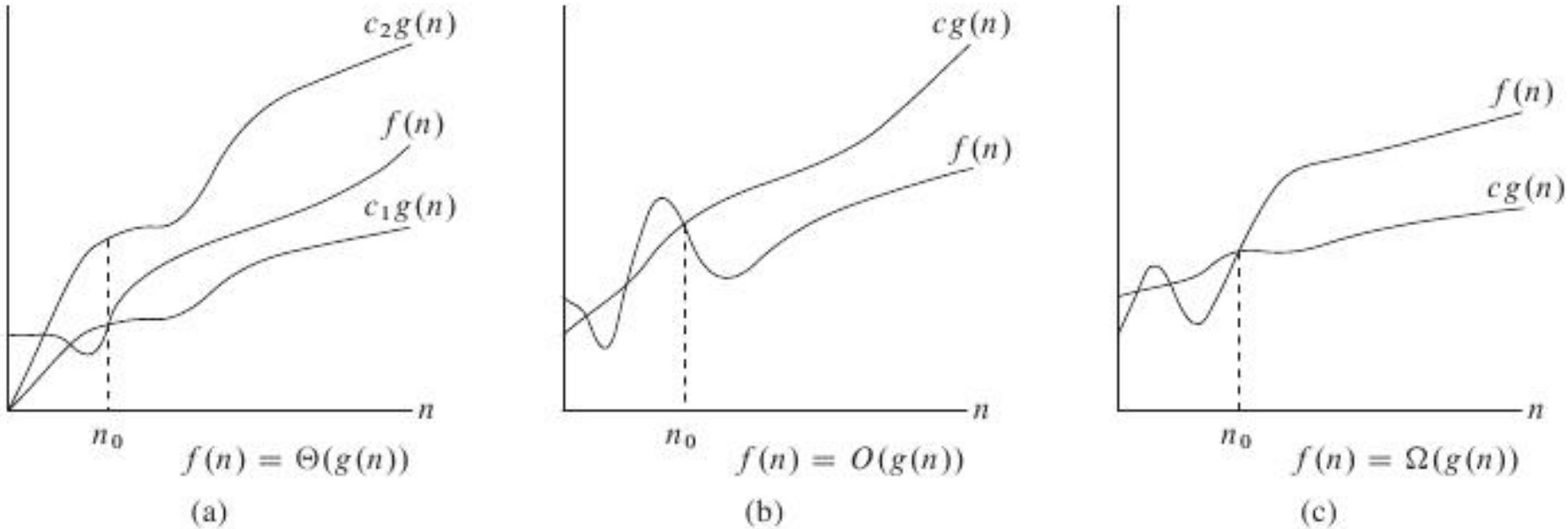


Figure 3.1 Graphic examples of the Θ, O, and Ω notations. In each part, the value of n_0 shown is the minimum possible value; any greater value would also work. **(a)** Θ-notation bounds a function to within constant factors. We write $f(n) = \Theta(g(n))$ if there exist positive constants n_0, c_1, and c_2 such that at and to the right of n_0, the value of $f(n)$ always lies between $c_1 g(n)$ and $c_2 g(n)$ inclusive. **(b)** O-notation gives an upper bound for a function to within a constant factor. We write $f(n) = O(g(n))$ if there are positive constants n_0 and c such that at and to the right of n_0, the value of $f(n)$ always lies on or below $cg(n)$. **(c)** Ω-notation gives a lower bound for a function to within a constant factor. We write $f(n) = \Omega(g(n))$ if there are positive constants n_0 and c such that at and to the right of n_0, the value of $f(n)$ always lies on or above $cg(n)$.

Asymptotic Notations (2)

Asymptotic Analysis

Notation	Relational Form	Limit Definition
$f(n)$ is $o(g(n))$	$f(n) \prec g(n)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$
$f(n)$ is $O(g(n))$	$f(n) \preceq g(n)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$ or 0
$f(n)$ is $\Theta(g(n))$	$f(n) \approx g(n)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$
$f(n)$ is $\Omega(g(n))$	$f(n) \succeq g(n)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$ or ∞
$f(n)$ is $\omega(g(n))$	$f(n) \succ g(n)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty.$

Solving Recurrences: Master Method

Let $a \geq 1$, $b > 1$ be constants and $T(n) = aT(n/b) + cn^k)$ for $n \geq 0$.

- case 1: $a > b^k$ then $T(n)$ is $\Theta(n^{\log_b a})$
- case 2: $a = b^k$ then $T(n)$ is $\Theta(n^k \log n)$
- case 3: $a < b^k$ then $T(n)$ is $\Theta(n^k)$

Classwork: Solve the follwing Recurrences:

- $T(n) = T(8n/11) + n$
- $T(n) = 16T(n/4) + n^2$

Table of Contents

1. Overview of the course
2. Recap: Algorithms
3. Algorithm Design Paradigms: Divide and Conquer
   - Lower bounds on Comparison-based sorting
   - Finding i^{th} smallest number
   - D & C: Long Integer Multiplications
4. References

Divide and Conquer : Sorting Algorithms

- Merge Sort:
  - Time complexity : $T(n) = 2T(n/2) + O(n) = \Theta(n \log n)$;
  - Space Complexity : $O(n)$
- Quick Sort:
  - Worst case Time complexity : $T(n) = T(0) + T(n-1) + O(n) = O(n^2)$,
  - Best case : $T(n) = 2T(n/2) + O(n) = O(n \log n)$,
  - Average case: $1/n \sum_{i=1}^{n} (T(i-1) + T(n-i) + n - 1) = \Theta(n \log n)$

Table of Contents

1. Overview of the course
2. Recap: Algorithms
3. Algorithm Design Paradigms: Divide and Conquer
   - Lower bounds on Comparison-based sorting
   - Finding i^{th} smallest number
   - D & C: Long Integer Multiplications
4. References

Lower bound on Comparison based Sorting (1)

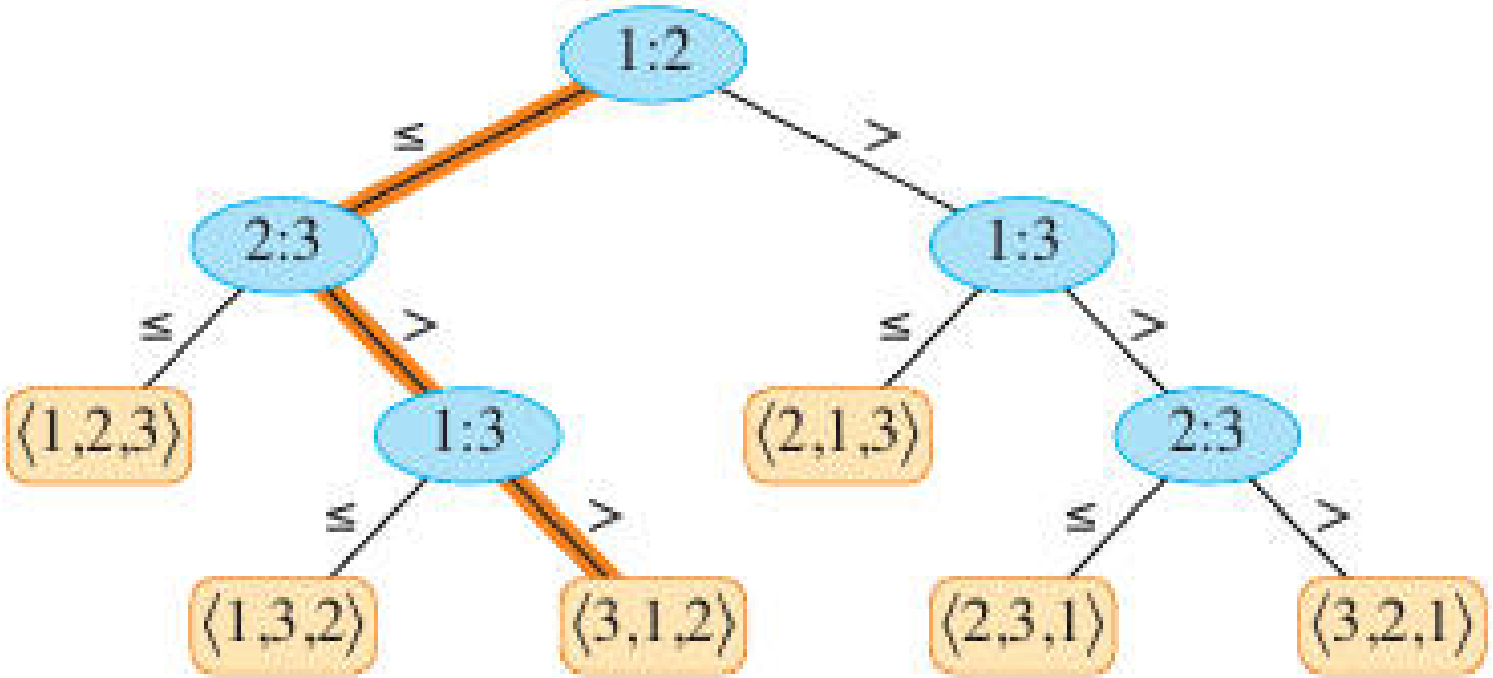
- The sorted order of determined by Insertion sort, Selection sort, Bubble sort, Quick sort, Merge sort, Heap sort are based only on **comparisons between the input elements**; Thus they are all **comparison sorts**
- Any **comparison sort** must make $\Omega(n \log n)$ comparisons in the worst case to sort n elements.
- Thus, **Merge sort and Heapsort are asymptotically optimal**, and no comparison sort exists that is faster by more than a constant factor.

Theorem: Any comparison sort algorithm requires $\Omega(n \log n)$ comparisons in the worst case

Lower bound on Comparison based Sorting (2)

Decision Tree model:

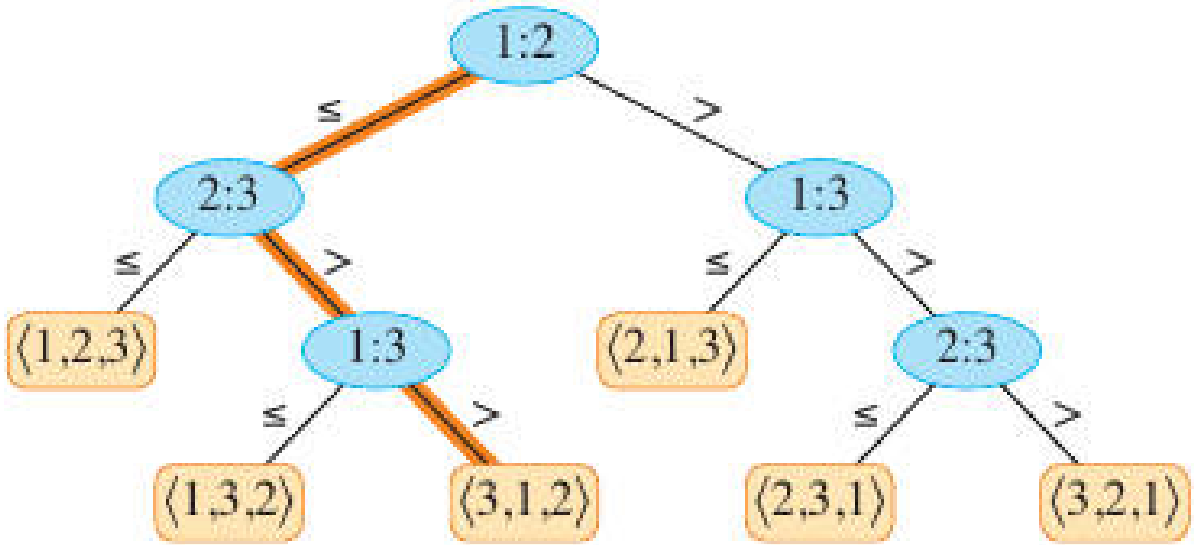
- A **decision tree** is a **full binary tree** (each node is either a leaf or has both children) that represents the comparisons between elements that are performed by a particular sorting algorithm operating on an input of a given size.
- The decision tree for insertion sort operating on three distinct elements.



Lower bound on Comparison based Sorting (3)

Decision Tree model:

- An internal node (shown in blue) annotated by $i : j$ indicates a comparison between a_i and a_j
- A leaf annotated by the permutation $< \cdots >$ indicates the ascending order of elements.
- The highlighted path indicates the decisions made when sorting the input sequence $< a_1 = 6, a_2 = 8, a_3 = 5 >$.
- Because the three input elements have $3! = 6$ possible permutations, the decision tree must have at least 6 leaves.



Lower bound on Comparison based Sorting : Proof (4)

- It is the height of the deceision tree (the number of comparisons required) that needs to be traversed from root to leaf to get the sorted order of a given input sequence of n elements for sorting.
- Consider a decision tree of height h with l reachable leaves corresponding to a comparison sort on n elements. We need to show $h \geq n \log n$
- Because each of the $n!$ permutations of the input appears as one or more leaves, we have $n! \leq l$.
- Since a binary tree of height h has no more than 2^h leaves, $n! \leq l \leq 2^h$
- Taking logarithms, $\log(n!) \leq h$, or, $h \geq \sum_{j=1}^{n} \log j = \Omega(n \log n)$, the lower bound on height of decision tree.

Note: $\sum_{j=1}^{n} \log j \leq \sum_{j=1}^{n} \log n = O(n \log n)$, also, $\sum_{j=1}^{n} \log j \geq \sum_{j=1}^{\lfloor n/2 \rfloor} \log(n/2) =$

$(\lfloor n/2 \rfloor) \log(n/2) = \lfloor n/2 \rfloor \log n - \lfloor n/2 \rfloor \implies \sum_{j=1}^{n} \log j = \Omega(n \log n)$

Approximate sum of $\log j$

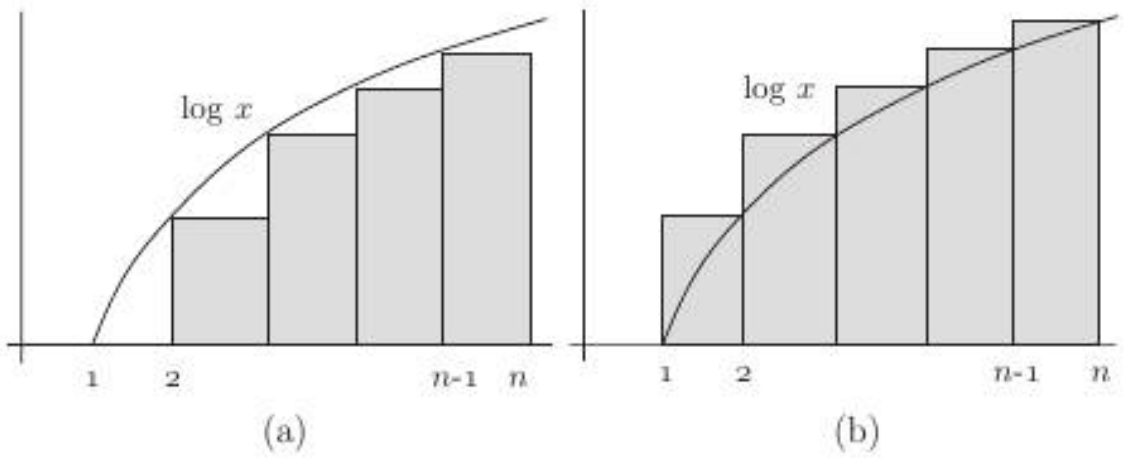


Fig. 2.3 Approximation of the sum $\sum_{j=1}^{n} \log j$.

- $\sum_{j=1}^{n} \log j = \log n + \sum_{j=1}^{n-1} \log j \leq \log n + \int_1^n \log x \; dx =$
  $\log n + n \log n - n \log e + \log e$
- $\sum_{j=1}^{n} \log j = \sum_{j=2}^{n} \log j \geq \int_1^n \log x \; dx = n \log n - n \log e + \log e$
- $n \log n - n \log e + \log e \leq \sum_{j=1}^{n} \log j \leq \log n + n \log n - n \log e + \log e$
- Thus, $\sum_{j=1}^{n} \log j = \Theta(n \log n)$

Table of Contents

1. Overview of the course
2. Recap: Algorithms
3. Algorithm Design Paradigms: Divide and Conquer
   - Lower bounds on Comparison-based sorting
   - Finding i^{th} smallest number
   - D & C: Long Integer Multiplications
4. References

Finding i^{th} smallest number: Selection Algorithm (1)

Problem: Given n elements, find the ith smallest number in $\Theta(n)$ time in the worst case.

Possible Solution: Sort the input in $O(n \log n)$ time using any comparison sort and report the ith element. But time complexity is $O(n \log n)$.

- Given 9 elements $I = \{3, 8, 1, 7, 2, 0, 11, 6, 5\}$ and an element p (pivot) from the set I, say $p = 6$, can you say if it is the $\lceil n/2 \rceil^{th}$ smallest element (median) of I in $\Theta(n)$ time?
- Count k, the number of elements less than or equal to 6 by scanning I only once. If $k = \lceil n/2 \rceil$ then 6 is median else not.
- **Conclusion:** Given a guess/ pivot x (6), we can report an yes/ no answer if it is the ith smallest number or not in $\Theta(n)$ time. Further, I is **partitioned** based on pivot x as in Quicksort.
  - Let A_l is the set of numbers less than the given guess / pivot x and A_r is the set of numbers greater than x
  - if $k = i$, then x is the ith smallest element
  - else if $k < i$, search the ith smallest element in A_r
  - else if $k > i$, search the ith smallest element in A_l

Finding i^{th} smallest number: Selection Algorithm (2)

- Given 9 elements $I = \{3, 8, 1, 7, 2, 0, 11, 6, 5\}$ and pivot x from the set I, say $x = 6$,find the median.
- To find $i = \lceil n/2 \rceil = \lceil 9/2 \rceil = 5^{th}$ smallest element from I, with $x = 6$, $A_l = \{3, 1, 2, 0, 5\}$, $A_r = \{8, 7, 11\}$
- Here $(k = 6) > (i = 5)$, thus search 5th smallest element in A_l. Note that 5th smallest element is 5 which is in A_l
- But what will be a nice guess/ pivot x?

A good pivot:

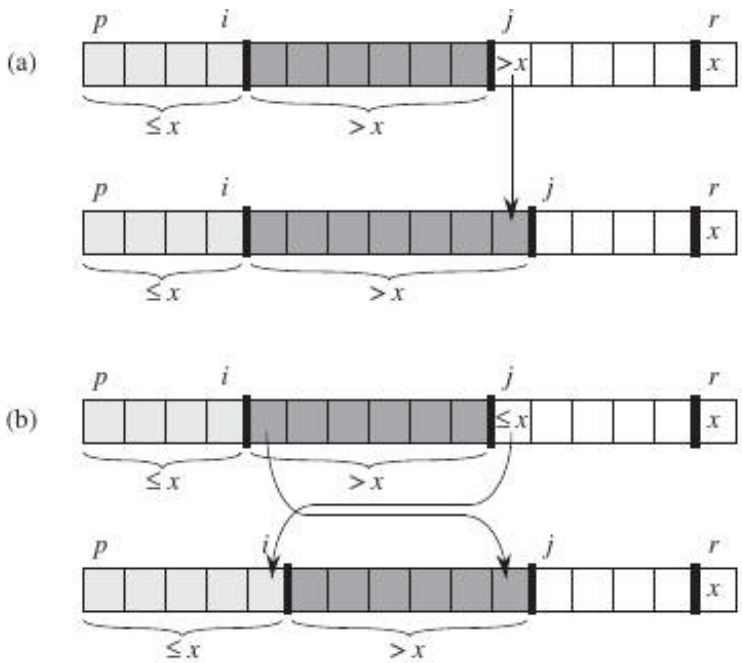
- Given n elements, break them into groups of 5 (why? will see later). Thus, there are $g = \lfloor n/5 \rfloor$ groups
- Sort each of g groups having 5 elements to get the **median of each group**. Let M is the set of g medians
- Find **median of medians** from M using **recursion** and let this median of median be the pivot x

Selection Algorithm: SELECT (3)

- Divide n elements of the input array into $g = \lfloor n/5 \rfloor$ groups of 5 elements each and at most one group made up of the remaining n mod 5 elements.
- Find the median of each of the g groups by first insertion-sorting the elements of each group and then picking the g medians from the sorted list of group elements.
- Use *SELECT* **recursively** to find the median/pivot x of the g medians found in step 2. (If there are an even number of medians, then by our convention, x is the lower median.)
- Partition the input array around the median-of-medians x using the modified version of *PARTITION* (as in quicksort). Let k be one more than the number of elements on the low side of the partition A_l, so that p is the kth smallest element and there are $n - k$ elements on the high side of the partition A_r.
- If $i = k$, then return x. Otherwise, use *SELECT* **recursively** to find the ith smallest element on the low side A_l if $i < k$, or the $(i - k)^{th}$ smallest element on the high side A_r if $i > k$.

Selection Algorithm: PARTITION (4)

```
PARTITION(A, p, r)
1  x = A[r]                          // the pivot
2  i = p − 1                         // highest index into the low side
3  for j = p to r − 1                // process each element other than the pivot
4      if A[j] ≤ x                   // does this element belong on the low side?
5          i = i + 1                 // index of a new slot in the low side
6          exchange A[i] with A[j]   // put this element there
7  exchange A[i + 1] with A[r]       // pivot goes just to the right of the low side
8  return i + 1                      // new index of the pivot
```



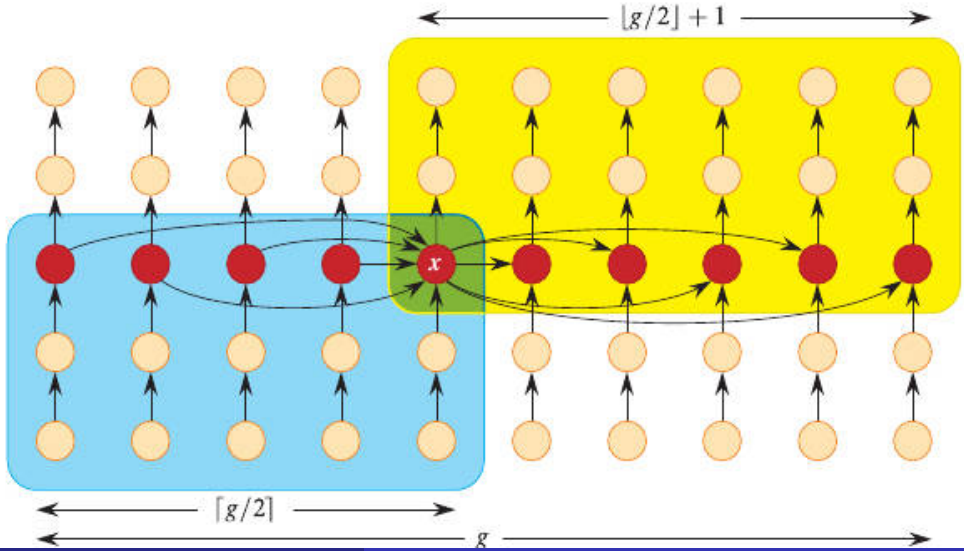
- Above is partition algorithm for Quicksort
- modify the *PARTITION* algorithm such that p is considered as pivot to this function (lines 1-2)

Selection Algorithm: SELECT (5)

```
SELECT(A, p, r, i)
 1  while (r − p + 1) mod 5 ≠ 0
 2      for j = p + 1 to r                    // put the minimum into A[p]
 3          if A[p] > A[j]
 4              exchange A[p] with A[j]
 5      // If we want the minimum of A[p : r], we're done.
 6      if i == 1
 7          return A[p]
 8      // Otherwise, we want the (i − 1)st element of A[p + 1 : r].
 9      p = p + 1
10      i = i − 1
11  g = (r − p + 1)/5                         // number of 5-element groups
12  for j = p to p + g − 1                    // sort each group
13      sort ⟨A[j], A[j + g], A[j + 2g], A[j + 3g], A[j + 4g]⟩ in place
14  // All group medians now lie in the middle fifth of A[p : r].
15  // Find the pivot x recursively as the median of the group medians.
16  x = SELECT(A, p + 2g, p + 3g − 1, ⌈g/2⌉)
17  q = PARTITION-AROUND(A, p, r, x)          // partition around the pivot
18  // The rest is just like lines 3–9 of RANDOMIZED-SELECT.
19  k = q − p + 1
20  if i == k
21      return A[q]                           // the pivot value is the answer
22  elseif i < k
23      return SELECT(A, p, q − 1, i)
24  else return SELECT(A, q + 1, r, i − k)
```

Selection Algorithm: Time complexity (1)

- The recurrence relation is : $T(n/5) + T(?) + \Theta(n)$
- Partitioning: $O(n)$, *SELECT* runs recursively to find x: $(T(n/5))$
- In Fig: arrows are from smaller value to larger values;x is median of medians, blue: elements $\leq x$; yellow: elements $> x$; white: can be in any partition
- At least half of $\lceil n/5 \rceil$ groups,i.e, $g/2$ groups contribute at least 3 elements that are less than x. Thus, the number of elements less than x is $3.\frac{1}{2}.n/5 = 3n/10$
- Since we compute **worst case time complexity**, need to see, recursion on which partition will take more time to end, i.e, $n - 3n/10 = 7n/10$
- Thus, $T(n/5) + T(7n/10) + \Theta(n)$



Selection Algorithm: Time complexity (2)

How to solve this recurrence? Use substitution method

- **Intuition:** Suppose in an algorithm for n elements, during each recursion we throw $1/3^{rd}$ of n elements and recurse only on the rest $2/3^{rd}$.
- Recursion in Iteration 1 happens on $2/3n$, in Iteration 2 on $2/3.2/3n = (2/3)^2 n$, in Iteration 3 on $(2/3)^3 n$ and so on...
- if the algo spends only constant time in for every element in each iteration, then total time is $cn + 2/3cn + (2/3)^2 cn + \cdots$ is $c\sum_{j=0}^{\infty}(2/3)^j n = c\frac{1}{(1-2/3)}n = 3cn = \Theta(n)$; as infinite geometric series.
- **Conclusion:** The intuitive idea for substitution method is thus $\Theta(n)$, when we discard a fraction of elements in recursion.

Selection Algorithm: Time complexity (3)

Solving $T(n/5) + T(7n/10) + \Theta(n)$

- Prove that $T(n) \leq cn$ for some suitably large constant $c > 0$ and all $n > 0$. Substituting this inductive hypothesis into the right-hand side of recurrence and assuming $n \geq 5$

$$\begin{aligned} T(n) &\leq c(n/5) + c(7n/10) + \Theta(n) \\ T(n) &\leq 9cn/10 + \Theta(n) \\ &= cn - cn/10 + \Theta(n) \\ &\leq cn \end{aligned}$$

- c is chosen large enough suc that $c/10$ dominates the upper-bound constant hidden by the $\Theta(n)$
- Thus, $T(n/5) + T(7n/10) + \Theta(n) = O(n)$

Why group size is 5?

- Look at the Recurrence relation, for group size 3:
  $T(n/3) + T(?) + \Theta(n)$
- Look at the Recurrence relation, for group size 7:
  $T(n/7) + T(?) + \Theta(n)$

Homework: Above

Table of Contents

1. Overview of the course
2. Recap: Algorithms
3. Algorithm Design Paradigms: Divide and Conquer
   - Lower bounds on Comparison-based sorting
   - Finding i^{th} smallest number
   - D & C: Long Integer Multiplications
4. References

Long Integer multiplications

Problem statement: Given two n-digit intgers A and B multiply to get the product in C, given n is very large as in crytographic applications.
Naive algorithm: Requires n^2 multiplications and n^2 additions. Thus Time complexity is $\Theta(n^2)$.
Want to improve the quadratic time complexity

- Let $m = n/2$. Let $w = A[n-1..m]$; $x = A[m-1..0]$ and $y = B[n-1..m]$, $z = B[m-1..0]$.
- Let w, x, y, z are $n/2$ digit numbers. Then A and B arise

$$A = w10^m + x$$
$$B = y10^m + z,$$

and their product is

$$mult(A, B) = mult(w, y)10^{2m} + (mult(w, z) + mult(x, y))10^m + mult(x, z)$$

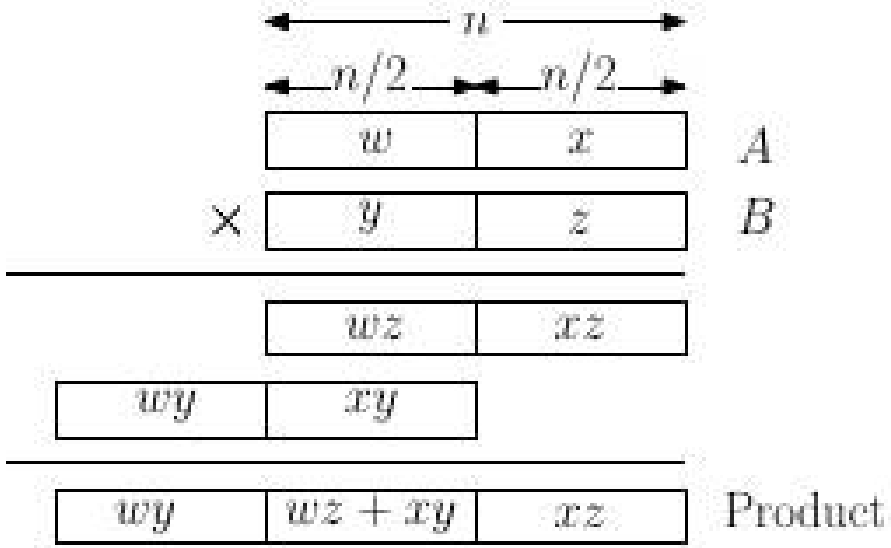
Long Integer multiplications (contd.)

- Let $m = n/2$. Let $w = A[n-1..m]$; $x = A[m-1..0]$ and $y = B[n-1..m]$, $z = B[m-1..0]$.
- Let w, x, y, z are $n/2$ digit numbers. Then A and B arise

$$A = w10^m + x$$
$$B = y10^m + z,$$

and their product is

$$mult(A, B) = mult(w, y)10^{2m} + (mult(w, z) + mult(x, y))10^m + mult(x, z)$$



Long Integer multiplications : Time complexity

- Multiplying by 10^m is simply shifting the number by m positions to the right; $\Theta(m)$ time
- The additions involve numbers involving roughly $n/2$ digits, and so they take $\Theta(n)$ time each.
- Thus, multiplication of two long integers results in **four products on integers of roughly half the length of the original**, and a **constant number of additions and shifts**, each taking $\Theta(n)$ time.

$$T(n) = 1; \text{if n=1}$$
$$T(n) = 4T(n/2) + n; \text{otherwise}$$

- This is $\Theta(n^2)$

Can we reduce number of multiplications(currently 4) by algebraic trick? May increase number of additions/subtractions(currently 3)

Long Integer multiplication: Faster D & C strategy

the trick:

$$\begin{aligned} C &= mult(w, y) = wy, D = mult(x, z) = xz \\ E &= mult((w + x), (y + z)) - C - D \\ &= (wy + wz + xy + xz) - wy - xz = (wz + xy) \\ mult(A, B) &= C.10^{2m} + E.10^{m} + D \end{aligned}$$

- 3 multiplications, 4 addtions, 2 subtractions with $n/2$ digits and m shift operations, ie, add/sub/shift takes $\Theta(n)$ time
- **Time complexity:** $T(n) = 3T(n/2) + n = \Theta(n^{\log 3}) = \Theta(n^{1.585})$
- Suppose **Faster Algo** has a overhead of 5 times than Normal D & C, then for larga n, check for which value of n, $5n^{1.585} < n^2$, then it will be wiser to apply Faster algorithm. Roughly around 40

Table of Contents

1. Overview of the course
2. Recap: Algorithms
3. Algorithm Design Paradigms: Divide and Conquer
   - Lower bounds on Comparison-based sorting
   - Finding i^{th} smallest number
   - D & C: Long Integer Multiplications
4. References

References

- Lecture notes on Design and Analysis of Algorithm , David Mount, 2015, Lecture 3, Lecture 28(Select Algorithm), Lecture 29 (Long Integer Multiplication)
- Introduction to Algorithms, T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein,Prentice Hall India, Fourth Edition, Chapter 9.3 (Select Algorithm)